

การออกแบบซอฟต์แวร์ (Software Design)

รายชื่อสมาชิกกลุ่ม

1. อรปรียา อาสนทอง	6605001699
2. อภิญญา วงษ์ศรี	6605005344
3. กิมทีนา ลี	6705006069
4. สุทธิดา ทรงรัมย์	6605005716
5. วิญญู สายแสง	6805007405

1.1 ความหมายของการออกแบบซอฟต์แวร์

กระบวนการกำหนดโครงสร้าง องค์ประกอบ และรายละเอียดของระบบซอฟต์แวร์ เพื่อให้สามารถพัฒนาและทำงานได้ตามความต้องการของผู้ใช้

เป็นขั้นตอนที่เชื่อมโยงระหว่างการวิเคราะห์ความต้องการ (Requirements Analysis) กับการพัฒนาโปรแกรมจริง

วัตถุประสงค์หลัก:

- ✓ ออกแบบข้อมูล ฐานข้อมูล และส่วนติดต่อผู้ใช้ (UI)
- ✓ กำหนดสถาปัตยกรรมของระบบ และแบ่งระบบออกเป็นโมดูลย่อย
- ✓ ทำให้ซอฟต์แวร์สามารถบำรุงรักษาและขยายได้ง่าย

1.2 กระบวนการออกแบบซอฟต์แวร์ (ส่วนที่ 1)

1) การออกแบบสถาปัตยกรรม (Architectural Design)

กำหนดโครงสร้างโดยรวมของระบบ ความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ และรูปแบบสถาปัตยกรรมที่เหมาะสม เช่น Client-Server หรือ Microservices

2) การออกแบบข้อมูล (Data Design)

กำหนดโครงสร้างข้อมูล ฐานข้อมูล และความสัมพันธ์ของข้อมูลภายในระบบ เพื่อให้การจัดเก็บและเรียกใช้งานมีประสิทธิภาพสูงสุด

3) การออกแบบส่วนประกอบ (Component Design)

แบ่งระบบออกเป็นโมดูล คลาส หรือฟังก์ชัน พร้อมกำหนดหน้าที่และการทำงานของแต่ละส่วนอย่างชัดเจน

1.2 กระบวนการออกแบบซอฟต์แวร์ (ส่วนที่ 2)



4) การออกแบบส่วนติดต่อผู้ใช้ (UI Design)

ออกแบบหน้าจอ การโต้ตอบ (Interaction) และประสบการณ์ของผู้ใช้ (User Experience) ให้ใช้งานง่าย สะดวก และตรงตามความต้องการของผู้ใช้งานจริง



5) การตรวจสอบและปรับปรุงแบบออกแบบ (Design Review)

ประเมินความถูกต้อง ประสิทธิภาพ และความเหมาะสมของแบบออกแบบก่อนเริ่มลงมือพัฒนาจริง เพื่อลดข้อผิดพลาดในขั้นตอนถัดไป

1.3 คุณภาพและการประเมินคุณภาพงานออกแบบ

คุณลักษณะ	คำอธิบาย
ความถูกต้อง (Correctness)	แบบออกแบบตอบสนองต่อความต้องการของผู้ใช้
ความเข้าใจง่าย	นักพัฒนาสามารถอ่านและทำความเข้าใจได้ง่าย
ความยืดหยุ่น และ การบำรุงรักษา	รองรับการเปลี่ยนแปลง และแก้ไข ปรับปรุงได้ง่าย
การนำกลับมาใช้ใหม่ & ประสิทธิภาพ	นำองค์ประกอบไปใช้ซ้ำได้ และใช้ทรัพยากรอย่างเหมาะสม

วิธีการประเมินคุณภาพ

→ การตรวจสอบแบบออกแบบ (Design Review)

→ การทดสอบต้นแบบ (Prototype Testing)

→ การใช้ตัวชี้วัดทางซอฟต์แวร์ (Software Metrics)

→ การประเมินโดยผู้เชี่ยวชาญ (Expert Evaluation)

1.4 แนวคิดด้านการออกแบบซอฟต์แวร์ (ส่วนที่ 1)

1) Abstraction

การซ่อนรายละเอียดที่ไม่จำเป็น และแสดงเฉพาะส่วนสำคัญของระบบ เพื่อลดความซับซ้อนในการทำความเข้าใจ

2) Modularity

การแบ่งระบบออกเป็นโมดูลย่อย ๆ เพื่อช่วยลดความซับซ้อน ทำให้ง่ายต่อการพัฒนา ทดสอบ และบำรุงรักษา

3) Encapsulation

การรวมข้อมูลและพฤติกรรมไว้ภายในหน่วยเดียวกัน พร้อมทั้งซ่อนรายละเอียดภายใน ไม่ให้เข้าถึงโดยตรงจากภายนอก

1.4 แนวคิดด้านการออกแบบซอฟต์แวร์ (ส่วนที่ 2)

4) Separation of Concerns

การแยกหน้าที่ของแต่ละส่วนออกจากกันอย่างชัดเจน เพื่อไม่ให้โค้ดมีความเกี่ยวพันกันจนเกินไป (Loose Coupling)

5) Coupling และ Cohesion

Coupling: ระดับการพึ่งพาระหว่างโมดูล ควรมีค่าต่ำ
Cohesion: ความสัมพันธ์ภายในโมดูล ควรมีค่าสูง

6) Refactoring

การปรับปรุงโครงสร้างของโค้ดโปรแกรมให้ดีขึ้น โดยไม่เปลี่ยนแปลงพฤติกรรมหรือผลลัพธ์การทำงานเดิมของระบบ



1.5 แบบจำลองการออกแบบซอฟต์แวร์

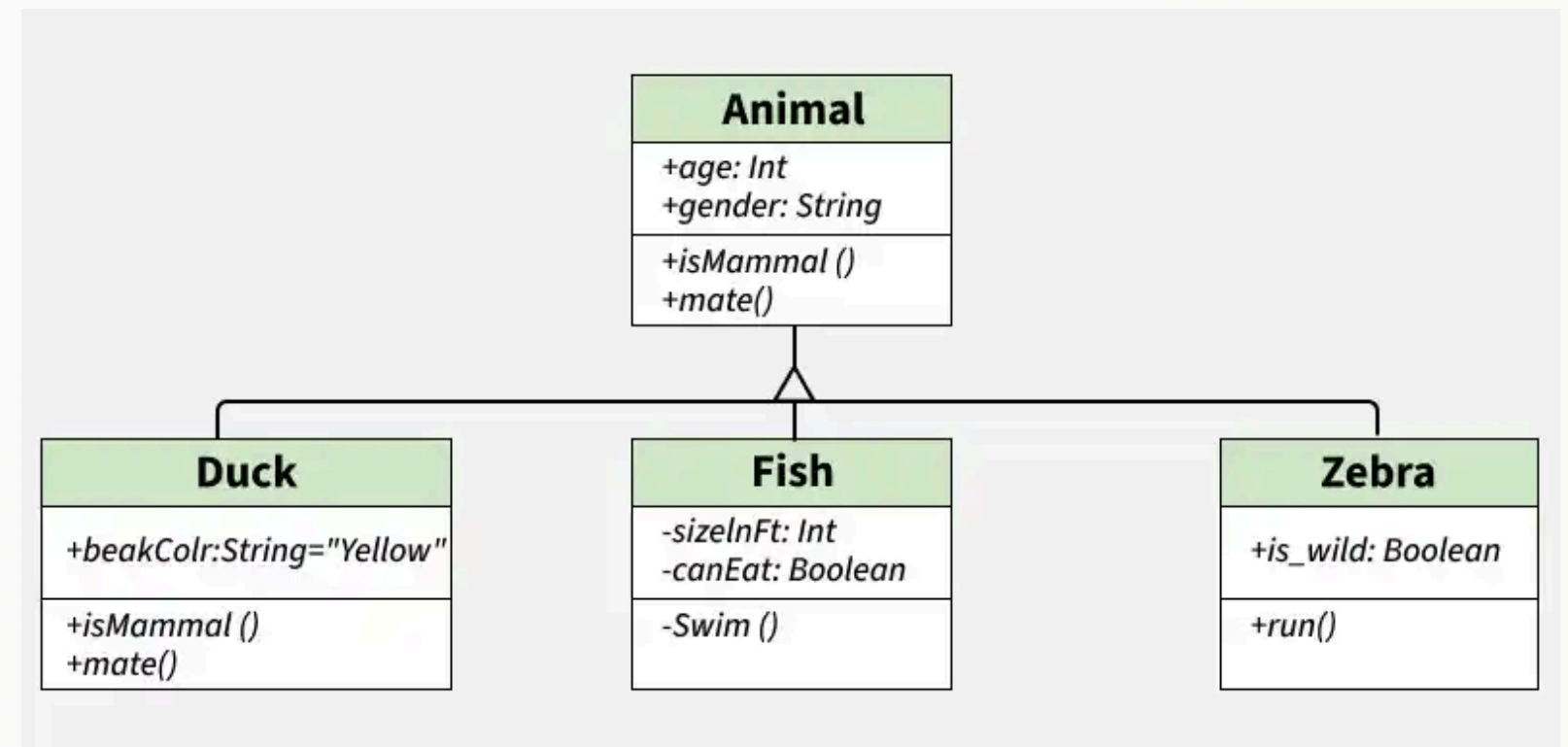
เป็นเครื่องมือที่ใช้แสดงโครงสร้างและพฤติกรรมของระบบ



1.5.1 แบบจำลองเชิงโครงสร้าง (Structural Model)

ใช้แสดงโครงสร้างของระบบและความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ

ตัวอย่างDiagram: Class Diagram, Object Diagram, Package Diagram, Component Diagram และ Deployment Diagram



Class Diagram

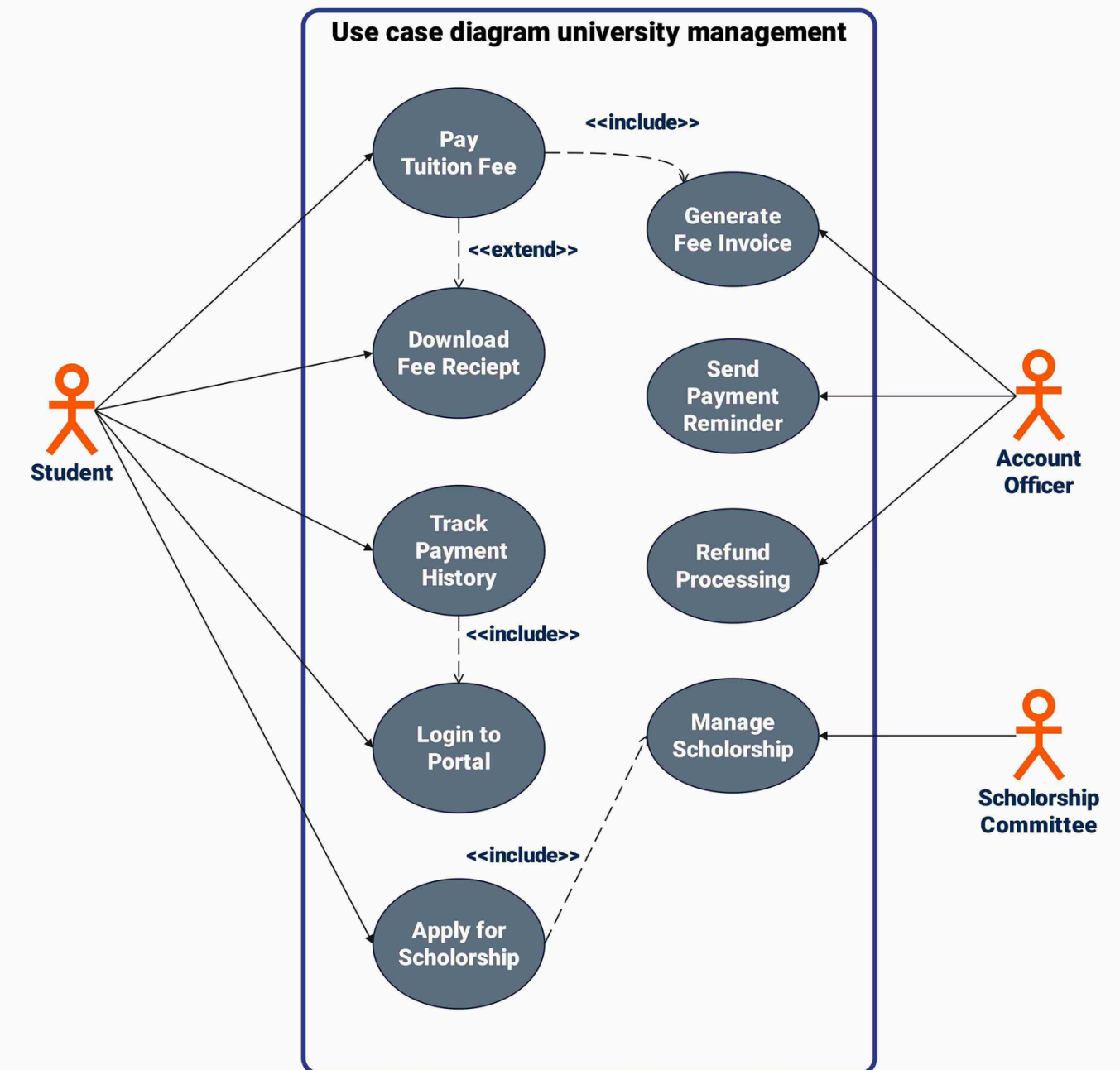
ตัวอย่าง Class Diagram แสดงโครงสร้างคลาส
คุณสมบัติ และความสัมพันธ์เชิงสืบทอด
(Inheritance) ของสัตว์ประเภทต่างๆ

1.5.2 แบบจำลองเชิงพฤติกรรม (Behavioral Model)

ใช้แสดงพฤติกรรมและการทำงานของระบบตามเหตุการณ์ต่างๆ

ตัวอย่างDiagram: Use Case Diagram, Activity Diagram, Sequence Diagram และ State Diagram

ตัวอย่าง Use Case Diagram แสดงการใช้งานระบบบริหารจัดการมหาวิทยาลัยระหว่างผู้ใช้งานและระบบ

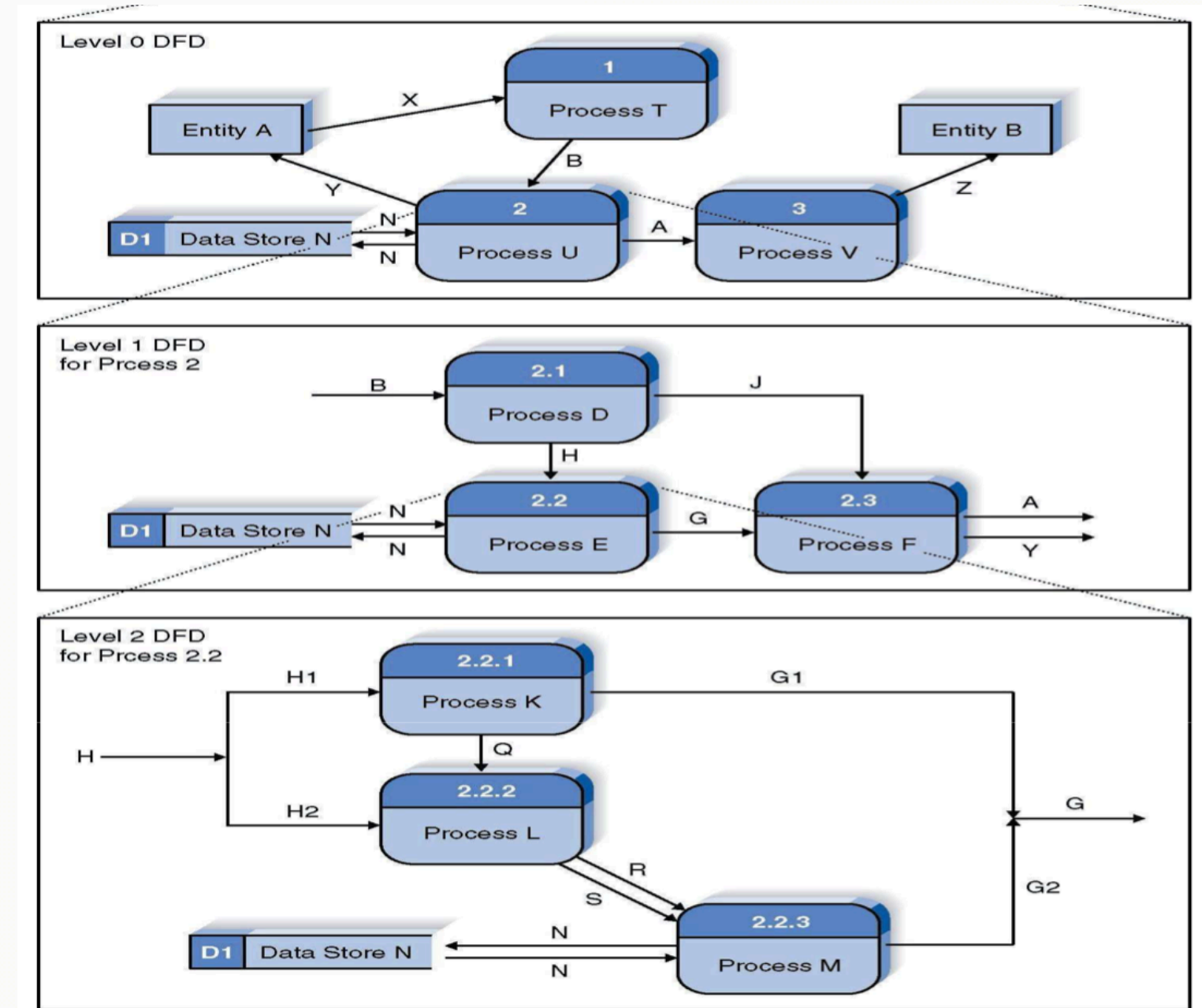


1.5.3 แบบจำลองเชิงข้อมูล (Data Model)

ใช้แสดงโครงสร้างข้อมูลและความสัมพันธ์ของข้อมูลภายในระบบ

ตัวอย่างDiagram: Entity Relationship Diagram (ERD) และ Data Flow Diagram (DFD)

ตัวอย่าง Data Flow Diagram (DFD) แสดงทิศทางการไหลของข้อมูลในระบบกระบวนการซื้อสินค้าและชำระเงิน



สรุป (Conclusion)

การออกแบบซอฟต์แวร์เป็นขั้นตอนสำคัญที่ช่วยกำหนดโครงสร้างและแนวทางการพัฒนาระบบให้มีประสิทธิภาพ โดยประกอบด้วยการออกแบบสถาปัตยกรรม ข้อมูล ส่วนประกอบ และ UI ซึ่งคุณภาพวัดได้จากความถูกต้อง ยืดหยุ่น และความสามารถในการบำรุงรักษา นอกจากนี้ แนวคิดอย่าง **Abstraction, Modularity และ Encapsulation** ช่วยลดความซับซ้อน พร้อมทั้งใช้แบบจำลองอย่าง **UML และ ERD** เพื่อให้ทีมพัฒนาเข้าใจตรงกันอย่างชัดเจน